

Estratégias de Caching

Lazy Loading, Write-Through e TTL

AWS Certified Developer Associate

Instrutor: Marlon Anesi

Lazy Loading (Cache-Aside)

Conceito:

Carregar dados no cache apenas quando necessário (sob demanda)

Como Funciona:

- 1. Aplicação busca dado no cache
- 2. Se encontrado (cache hit): retorna dado
- 3. Se não encontrado (cache miss): busca no database
- 4. Armazena dado no cache para próximas requests
- 5. Retorna dado para aplicação

Vantagens:

- Cache contém apenas dados requisitados (eficiente)
- Resiliente: se cache falhar, app continua funcionando
- Simples de implementar

Desvantagens:

- Cache miss penalty: primeira request é sempre lenta
- Dados stale: cache pode conter dados desatualizados
- Cache churn: muito misses podem sobrecarregar database

Write-Through

Conceito:

Atualizar cache sempre que o dado é escrito no database

Como Funciona:

- 1. Aplicação escreve dado no cache
- 2. Cache escreve dado no database (síncronamente)
- 3. Ambos cache e database ficam sincronizados

Vantagens:

- Dados sempre atualizados (consistentes)
- Elimina dados stale no cache
- Reads sempre rápidos (dados já estão no cache)

Desvantagens:

- Write penalty: toda escrita passa pelo cache e database
- Missing data: dado nunca lido ainda não está no cache
- Cache churn: dados raramente lidos ocupam espaço
- Maior latência em writes

Combinação com Lazy Loading:

- Melhor estratégia: combinar write-through com lazy loading
- Write-through mantém dados atualizados
- Lazy loading preenche cache com dados lidos

TTL - Time To Live

Conceito:

Definir tempo de expiração automática para dados no cache

Como Funciona:

- Ao adicionar dado no cache, define TTL em segundos
- Após TTL expirar, dado é automaticamente removido
- Próximo acesso será cache miss (forçando atualização)

Vantagens:

- Evita dados stale: refresh automático
- Gerenciamento automático de memória
- Flexível: TTL diferente por tipo de dado
- Simples de implementar

Desvantagens:

- Difícil definir TTL ideal para cada tipo de dado
- TTL muito curto: muitos cache misses
- TTL muito longo: dados podem ficar stale

Recomendações de TTL:

- Dados que mudam pouco: TTL de horas (ex: 3600s = 1h)
- Dados que mudam frequentemente: TTL de minutos (ex: 300s = 5min)
- Dados críticos: TTL curto + invalidação manual

Comparação e Melhores Práticas

Estratégia	Quando Usar	Principal Vantagem
Lazy Loading	Read-heavy workloads	Cache eficiente (só dados usados)
Write-Through	Dados devem estar sempre atualizados	Elimina dados stale
TTL	Complementar outras estratégias	Refresh automático
Lazy + Write-Through	Maioria dos casos (recomendado)	Melhor consistência e performance

Melhores Práticas:

- Combine estratégias: Lazy Loading + Write-Through + TTL
- Defina TTL apropriado baseado na frequência de mudança dos dados
- Implemente cache invalidation para dados críticos
- Monitore cache hit ratio (objetivo: > 80%)
- Use namespaces para organizar chaves (ex: user:123, product:456)
- Implemente circuit breaker para proteger database em caso de cache failure
- Adicione jitter ao TTL para evitar thundering herd (múltiplos expiram ao mesmo tempo)
- Configure alarms para monitorar evictions e memory pressure

Pontos-Chave - Parte 1



1. Lazy Loading

Cache sob demanda (apenas dados requisitados)

Vantagem: eficiente, resiliente

Desvantagem: cache miss penalty, dados stale

2. Write-Through

Atualiza cache em toda escrita no database

Vantagem: dados sempre atualizados

Desvantagem: write penalty, cache churn

3. TTL

Expiração automática de dados

Evita dados stale e gerencia memória

Desafio: definir TTL ideal por tipo de dado

Pontos-Chave - Parte 2

4. Melhor Estratégia

Combinar: Lazy Loading + Write-Through + TTL

Obtém benefícios de todas as estratégias

5. Monitoramento

Cache hit ratio: objetivo > 80%

Evictions: indica memory pressure

Latência: reads devem ser sub-millisecond

LEMBRE-SE: Lazy Loading carrega sob demanda. Write-Through mantém consistência. TTL evita dados stale. Combine as três para melhor resultado!